

Inspiratoria — Informe técnico A → Z

Plataforma multi-tenant de programas de mentoría con matching inteligente, portal de participantes sincronizado en tiempo real y consola de PM (Project Manager) para diseñar, ejecutar y cerrar programas de principio a fin.

Stack: Django 5 + FastAPI + Next.js 14 + PostgreSQL + Socket.IO + Gemini. **Producción:** inspiratoria.aplily.com **Auto-deploy:** Render (rama `main`). **Última actualización:** 2026-05-09.

Tabla de contenidos

- 1. Estructura del repositorio
- 2. Backend (Django + FastAPI)
- 3. Frontend (Next.js)
- 4. Modelo de datos
- 5. Roles y autenticación
- 6. Flujos de dominio
- 7. Matching inteligente
- 8. Portal de participante (`/p/{code}`)
- 9. Despliegue
- 10. Variables de entorno
- 11. Comandos y scripts
- 12. Histórico de commits relevantes
- 13. Glosario

1. Estructura del repositorio

inspiratoria/	
├─ backend/	Django 5 + FastAPI (puerto 8001)
├─ mentorloop_clone/	Settings, ASGI, URLs raíz
├─ api/	Router FastAPI principal (mount /api)
├─ companies/	Multi-tenant: Company + User custom + OTP/TOTP
└─ programs/	Dominio principal: Program, Participant, Match, Activity,
Milestone	
└─ services/	
└─ intelligent_matching.py	Algoritmo 6D mentor-mentee
└─ invitations/	Tokens, onboarding 6 pasos, LinkedIn OAuth, Gemini
└─ seed_*.py	Scripts de carga de datos demo
└─ requirements.txt	
├─ frontend/	Next.js 14 + TS + Tailwind (puerto 3000)
└─ src/app/	App router (rutas)
└─ src/components/	UI compartida (Sidebar, dashboards, modales, chat...)
└─ package.json	
├─ docs/	Diagramas y documentación adicional
├─ scripts/	Utilidades (seed, deploy, checks)
├─ render.yaml	Blueprint de despliegue (backend + frontend + Postgres)
├─ start_all.sh	Levanta backend + frontend en local
├─ start_backend.sh	Solo backend

— *.md	~20 documentos de arquitectura/UX/operación
— PROJECT_OVERVIEW.md	(este archivo)

Documentación relevante en raíz

Archivo	Contenido
README.md	Quick start y resumen general
BACKEND_STATUS.md	Estado funcional del backend
DASHBOARD_SYSTEM.md	Arquitectura de dashboards por rol
ROLES_ARCHITECTURE.md	Mapa completo de roles y permisos
ONBOARDING_SYSTEM.md	Onboarding LinkedIn + Gemini
DEPLOYMENT.md / RENDER_SETUP.md	Guía de despliegue Render
UX_IMPROVEMENTS_LOG.md	Cambios UX/UI recientes
TESTING_GUIDE.md	Procedimientos de prueba

2. Backend (Django + FastAPI)

2.1. Stack

Componente	Versión	Uso
Python	3.10.16	Runtime
Django	5.x	ORM, auth, admin, migraciones
FastAPI	0.104+	Endpoints REST modernos sobre Django
Uvicorn	latest	Servidor ASGI
python-socketio	latest	Chat en tiempo real
psycopg2-binary	latest	Driver PostgreSQL
pyotp	latest	TOTP (2FA)
qrcode	latest	QR para enrolar TOTP
google-generativeai	0.x	Gemini 2.5 Flash
pandas + openpyxl	latest	Importar/exportar Excel
dj-database-url	latest	Config DB desde DATABASE_URL

2.2. Entry point ASGI — backend/mentorloop_clone/asgi.py

/socket.io	→ Socket.IO (chat, presencia)
/api/*	→ FastAPI (router en backend/api/routes.py)
/static/*	→ Archivos estáticos (avatares base64 + assets)
/*	→ Django ASGI (admin, OAuth, etc.)

2.3. Apps Django

companies/ — multi-tenant + auth

- **Company** (PK UUID): nombre, slug, RUT/tax_id, contacto, plan (trial|starter|growth|enterprise), estado (pending|trial|active|suspended|cancelled), branding (logos + colores), assigned_pm (FK User).
- **User** (extiende AbstractUser, PK UUID):
 - **Roles:** superadmin, admin_root, inspiratoria_admin, coordinator, project_manager, billing, client, admin, facilitator_internal, facilitator_inspiratoria, mentor, mentee, participant, participant_cell.
 - **Perfil base:** full_name, phone, position, department, linkedin_url, headline, skills, bio.
 - **Wizard de mentor (4 pasos):** mentor_topics, mentor_objectives, mentor_style, experience_level, experience_area, mentor_profile_step.
 - **Wizard de mentee (4 pasos):** mentee_goals, mentee_interests, mentee_challenges, mentee_expectations, preferred_mentor_style, session_format_preference, mentee_profile_step.
 - **Permisos granulares:** can_manage_clients, can_manage_programs, can_manage_users, can_manage_activities, can_execute_matches, can_view_reports, can_close_programs, can_manage_alerts.
 - **Auth:** otp_code, otp_expires_at, totp_secret, totp_enabled, is_account_activated, portal_code (8 chars únicos para /p/{code}).
- **Helpers:** prepare_user_secure_access(user, send_email) genera OTP y envía mail.

Endpoints clave (montados bajo /api/companies):

Método	Ruta	Descripción
POST	/onboarding/step1	Crea empresa
POST	/onboarding/step2/{company_id}	Crea super admin
POST	/onboarding/step3/{user_id}	Cierra onboarding
POST	/login/otp	Solicita OTP por email
POST	/verify-otp	Verifica OTP y devuelve token
POST	/totp/setup /totp/verify	2FA opcional
GET	/portal/{portal_code}	Datos del portal del participante
GET	/portal/{portal_code}/mentees	Mentees asignados a un mentor
GET	/portal/{portal_code}/badges	Insignias
GET	/portal/{portal_code}/activities	Actividades del portal

programs/ — dominio principal

Modelos (resumen):

- **Program:** name, description, theme, company, status ∈ {designed, ready_for_execution, in_execution, under_review, closed}, requires_certification, certification_rules.
- **ProgramTemplate** (en api/): plantillas reutilizables con modules, milestones, tags, duration.
- **ProgramParticipant / Participant:** relación usuario↔programa con role, status ∈ {pending,

active, removed}, joined_at, requires_match.

- **Match:** mentor, mentee, program, score, status, único por terna.
- **Milestone:** hitos por match con status, due_date, order.
- **Sentiment:** rating 1–5 + nota por sesión.
- **Notification:** notificaciones in-app con tipo, link y read flag.
- **Activity:** type ∈ {training, event}, modality ∈ {online, presencial, hibrido}, start_date, end_date, meeting_url, location_address, target_role, is_mandatory, is_certificate_issued.
- **Content:** materiales por actividad (orden, publicado, materials_url).
- **Survey:** type ∈ {modulo, satisfaccion}, JSON de preguntas, métricas de respuesta.
- **Alert:** alertas operativas (activity_delayed, low_confirmation, low_attendance, pending_surveys, match_pending) con flujo active → in_progress → resolved | dismissed.

Endpoints clave (/api/programs):

Método	Ruta	Descripción
GET / POST	/	Listar / crear programas
GET	/ {id}	Detalle del programa con actividades
PATCH	/ {id} / status	Transiciones de estado
GET	/ {id} / participants	Participantes del programa
POST	/ {id} / participants	Agregar participante (admin)
POST	/ {id} / participants / {pid} / resend-invitation	Reenviar OTP
GET	/ {id} / public-info	Info pública (sin auth)
POST	/ {id} / self-enroll	Auto-enrolamiento público
POST	/ {id} / smart-match	Ejecuta matching IA
GET	/ my-programs / {user_id}	Programas del usuario actual
CRUD	/ program-templates	Plantillas reutilizables

invitations/ — invitaciones, onboarding, LinkedIn + IA

- **PendingInvitation:** email, role, token (64 chars), company, program, invited_by, status ∈ {pending, accepted, expired, cancelled}, expires_at (7 días).
- **OnboardingProgress:** progreso por usuario en wizard de 6 pasos (validate → account → linkedin → profile → role → complete).
- **Servicios:**
 - LinkedInAuthService: flujo OAuth 2.0 → trae perfil real.
 - NeuralmorphicAIService (Gemini 2.5 Flash): extrae topics/skills/bio del perfil LinkedIn y sugiere texto de presentación.

Endpoints (/api/invitations):

Ruta	Descripción
/invite	Envía invitación (token + email)
/validate	Valida token (paso 1)

/create-account	Paso 2: crear cuenta con datos del invite
/linkedin/auth-url	Inicia OAuth LinkedIn (paso 3)
/linkedin/callback	Procesa callback + extracción Gemini
/complete-profile	Paso 4–5: revisar/editar perfil
/complete-onboarding	Paso 6: activa cuenta y notifica

api/ — capa FastAPI

- routes.py monta routers de companies/programs/invitations.
- Schemas Pydantic centralizados para validación.
- /api/health y /api/docs (OpenAPI).

2.4. Email pipeline

- SMTP Gmail (smtp.gmail.com:587).
- Remitente: Inspiratoria <macarena@inspiratoria.org>.
- Templates en español (HTML inline) para:
 - OTP de acceso (prepare_user_secure_access).
 - Invitaciones a programas (send_participant_access_email).
 - Notificaciones de sesiones (mentor/mentee).
- Reenvío bajo demanda desde el botón “Reenviar invitación” en la consola PM.

2.5. Scripts y management commands

- seed_intelligent_match_demo.py — 10 mentores + 10 mentees con perfiles ricos para probar el algoritmo de matching.
- seed_all_roles.py — Pobra todos los roles del sistema.
- clear_all_keep_admin.py — Limpia DB conservando admin root.
- create_mentor_test.py — Mentor de prueba.
- Comandos Django bajo */management/commands/.

3. Frontend (Next.js)

3.1. Stack

Tech	Versión	Uso
Next.js	14.1.0	App router
React	18.2.0	UI
TypeScript	5.2.2	Tipado
Tailwind CSS	3.3.5	Estilos utilitarios
Chart.js + react-chartjs-2	4.5.1	Métricas
FullCalendar	6.1.19	Calendario / cronograma
socket.io-client	4.8.1	Chat real-time

TipTap	3.22.3	Rich text
axios + SWR	latest	HTTP y cache
framer-motion	latest	Animaciones
lucide-react	latest	Iconografía
jspdf + xlsx	latest	Export PDF/Excel

3.2. Rutas (src/app/)

```

/ Landing público
/login Login email/OTP estándar
/login/admin Login admin + OTP + TOTP
/register Registro
/activate/[token] Activación por token
/onboarding Wizard de 6 pasos (LinkedIn + IA)
/onboarding/linkedin-callback Callback OAuth
/enroll/[programId] Auto-enrolamiento público a programa

/dashboard Consola admin (rol-aware)
  ├── accounts/[id] Detalle de cuenta + sub-vistas internas
  ├── activities Gestión de actividades
  ├── alerts Bandeja de alertas operativas
  ├── analytics KPIs y métricas
  ├── billing Suscripciones / cobros
  ├── calendar Vista calendario / cronograma
  ├── chat Mensajería
  ├── clients/[id]/{billing|config|manage|programs|setup|users}
  ├── configuration Config sistema
  ├── ecosystem Ecosistema / partners
  ├── goals Objetivos / OKRs
  ├── intelligent-match UI del motor de matching IA
  ├── manage-programs CRUD de programas
  ├── matches Listado de matches
  ├── migration Herramientas de migración
  ├── notifications Centro de notificaciones
  ├── participants Gestión global de participantes
  ├── profile Perfil del usuario admin
  ├── programs/[id]/{activities|config|manage|participants|reports|training}
  ├── reports Reportes
  ├── sessions Sesiones de mentoría
  └── users Usuarios

/studio Studio (PM Console v2)
  └── [slug] Workspace por empresa
      ├── [section]
      ├── program/[programId] Diseño/edición del programa
      └── participant/[programId]/[role]/[participantId]

/p/[code]/[...section] Portal de participante (catch-all)
  Secciones: '', programa, progreso, modulos, actividades, participantes,
             hitos, ecosistema, perfil, mis-mentees, mi-mentor, sesiones,
             mi-red, mis-actividades, insignias, chat

/mentor[/program] Vista resumida del mentor
/mentee[/program] Vista resumida del mentee
/participant/[id] Detalle individual genérico
/core[/...] Páginas institucionales
/api/{core-welcome|studio-credentials|studio-inquiry} Route handlers Next

```

3.3. Componentes destacados

- Sidebar, TopNavbar — navegación y menú de usuario.
- DashboardSkeleton, DashboardStatsCards — esqueleto de carga + KPIs animados.
- MatchingEngine, MatchingEngineAI — UI del matching (manual vs Gemini).
- ParticipantModal, ProgramManagement — CRUD con UX consistente.
- ChatWindow (Socket.IO) y CalendarView (FullCalendar).
- dashboards/{Admin|Client|Facilitator|Mentor|Mentee}Dashboard — vistas por rol.

3.4. Patrones de diseño

- THEME_GRADIENTS (6 gradientes preset por temática).
- Sistema de **skeletons** reutilizables en el portal: SkelBlock, SkelCardGrid, SkelList, SkelDetailPage. Cero spinners morados — el primer paint replica la silueta final del portal.
- Indicador “Sync hh:mm” + auto-refresh on focus / visibilitychange para mantener el portal sincronizado con la consola PM sin recargar.
- Modales con tabs grandes (icon + label) — UX validada en el wizard de “Agregar participante” y “Crear nuevo / Buscar existente”.

4. Modelo de datos (resumen relacional)

Company 1—* User	
Company 1—* Program	
Program 1—* Activity 1—* Content	
Program 1—* Survey	
Program 1—* Alert	
Program 1—* ProgramParticipant *—1 User	
Program 1—* Match (mentor: Participant, mentee: Participant)	
Match 1—* Milestone	
Match 1—* Sentiment	
User 1—* Notification	
PendingInvitation *—1 Company / Program / User(invited_by)	
OnboardingProgress 1—1 User (con FK a PendingInvitation)	

5. Roles y autenticación

5.1. Jerarquía de roles

#	Rol	Alcance	Acceso por defecto
1	superadmin / admin_root	Plataforma	/dashboard (todo)
2	inspiratoria_admin	Multi-cliente	/dashboard
3	coordinator	1 empresa	/dashboard
4	client	Stakeholder	/dashboard (solo lectura)
5	admin	Empresa cliente	/studio
6	project_manager	Programa	/studio

7	facilitator_internal / facilitator_inspiratoria	Sesiones	/p/{portal_code}
8	mentor	Acompañamiento	/p/{portal_code}
9	mentee	Aprendizaje	/p/{portal_code}
10	participant / participant_cell	Eventos / célula	/p/{portal_code}

5.2. Login (OTP)

1. /login/admin → email.
2. Backend genera OTP de 4 dígitos, expira 15 min, lo envía por email.
3. Usuario ingresa OTP.
4. Backend devuelve {token, user, company, program_participant?, expires_at}.
5. Frontend guarda en localStorage, decide ruta según role + portal_code.

5.3. TOTP (opcional)

- Usuario habilita TOTP → backend genera totp_secret → se renderiza QR.
- En el siguiente login se exige código adicional.

5.4. Portales por código

- Cada User tiene portal_code único de 8 chars.
- Toda la experiencia del participante vive en /p/{portal_code}/..., por lo que la URL es shareable entre PM y participante sin exponer IDs.

6. Flujos de dominio

6.1. Crear programa

1. /dashboard/manage-programs → “Crear programa”.
2. Wizard de 2 pasos: plantilla + tema → empresa + reglas de certificación.
3. Programa nace con status=designed y queda listo para diseñar cronograma y participantes en /studio/{slug}/program/{id}.

6.2. Invitar participante (admin)

1. PM Console → tab **Participantes** → botón **Agregar**.
2. Modal con dos pestañas grandes:
 - **Buscar existente** (búsqueda con debounce 250 ms).
 - **Crear nuevo** (email + nombre + apellido + rol).
3. Toggle “Enviar invitación”: si está activo se envía OTP automáticamente.
4. Backend crea/recupera User, alta como ProgramParticipant (status=pending), genera token y dispara email vía prepare_user_secure_access.
5. Per fila aparece estado, fecha de invitación, botón **Reenviar OTP** y **Eliminar**.

6.3. Auto-enrolamiento público

1. Banner copiable en el tab Participantes con la URL /enroll/{programId}.

2. Página pública (sin auth):

- GET /api/programs/{id}/public-info (nombre, descripción, empresa, estado, accepting_enrollments).
- 4 cards de rol (Mentor / Mentee / Facilitador / Participante).
- Form: email + nombre + apellido.

3. POST /api/programs/{id}/self-enroll crea User si no existe, lo enrola como pending y manda OTP. Re-activa soft-deleted si aplica.

4. Pantalla de éxito con CTA a /login.

6.4. Cronograma

1. En PM Console → **Cronograma**.
2. **Asistente automático** sugiere fechas según duración del programa, hitos del template y cantidad de matches.
3. Cada actividad puede editarse manualmente (modal con start_date / end_date, modalidad, link de reunión).
4. La línea de tiempo refleja cambios al instante.
5. El portal lee las mismas fechas (programDetail.activities[].start_date) y las muestra en /p/{code}/modulos, /p/{code}/actividades y /p/{code}/progreso.

6.5. Sesiones de mentoría

- Mentor agenda sesión desde /p/{code}/sesiones (rol mentor).
- Mentee la ve en /p/{code}/sesiones (rol mentee) y /p/{code}/mi-mentor.
- Notas + estado de ánimo + temas + próximos pasos por sesión.
- IA (Gemini) sugiere agenda de la próxima sesión a partir de las notas.
- Email automático al mentee al agendar / al mentor al confirmar.

6.6. Insignias y progreso

- /p/{code}/insignias muestra logros conseguidos vs. globales.
- /p/{code}/progreso calcula timeline real con scheduledActivities[0] → startDate y scheduledEnds[last] → endDate. Si no hay cronograma, fallback a joined_at + duración del template.

7. Matching inteligente

Implementado en backend/programs/services/intelligent_matching.py.

7.1. Score final (suma ponderada)

Dimensión	Peso	Cálculo
Skills × Goals	25 %	mentor.skills ∩ mentee.goals/interests
Topics × Challenges	20 %	mentor.mentor_topics ∩ mentee.mentee_challenges/interests
Style fit	15 %	match entre mentor_style y preferred_mentor_style
Experience fit	15 %	tier gap por experience_level + experience_area

Objectives fit	15 %	<code>mentor.mentor_objectives ∩ mentee.mentee_expectations</code>
Domain fit	10 %	<code>similitud_{position/department/headline}</code>

7.2. Atenuación por completitud

Si `mentee_profile_step < 4` o `mentor_profile_step < 4`, se aplica un factor proporcional a la completitud del perfil para no premiar perfiles vacíos.

7.3. Capa Gemini (opcional)

Activable con `useAI=true`. Gemini 2.5 Flash recibe los dos perfiles y devuelve un **resumen ejecutivo** de por qué hace match (tres bullets), que se renderiza en la UI debajo del breakdown numérico.

7.4. UI

`/dashboard/intelligent-match:`

- Selector de programa, `topK`, `minScore`, `toggle IA`.
- Resultado por par: score total, breakdown por dimensión, keywords coincidentes, acción **Activar match** → cambia `Match.status = active` y notifica a ambos.

8. Portal de participante (`/p/{code}`)

Implementado en `frontend/src/app/p/[code]/[...section]/page.tsx` (~4.7 k líneas).

8.1. Sincronización en vivo

- `reloadProgramData({silent})` carga en paralelo: `/programs/{id} + /program-templates + /participants + /my-programs/{user_id}`.
- Listeners en `window.focus` y `document.visibilitychange` → refresco silencioso.
- Indicador “Sync hh:mm” + botón manual con animación spin.
- Resultado: cualquier cambio hecho en `/studio/...` aparece en el portal sin recargar.

8.2. Skeletons consistentes

- Pantalla inicial: skeleton del shell (sidebar + topbar + stats + grid). Elimina el destello morado que existía antes (`#7c3aed/onboarding`).
- Cada vista (programa, progreso, módulos, sesiones, red, actividades, mentor, insignias) usa `SkelDetailPage / SkelList / SkelCardGrid`.

8.3. Tab “Módulos” sincronizado con cronograma

- Cada actividad de un módulo se cruza con `programDetail.activities` por `name + module_id`.
- Si está agendada se muestra fecha/hora y modalidad.
- Si no, badge **Sin agendar**.
- Sección extra “**Actividades adicionales del cronograma**” lista las actividades que no pertenecen a ningún módulo del template (ej.: extras añadidos directamente en `/studio/.../cronograma`).

8.4. Wizard de perfil

- 4 pasos (mentor o mentee) con validación por paso.
- **Nombre y apellido** y **email de acceso** son **read-only**: vienen del invite. Aparece un campo opcional

email personal alternativo.

- Bloqueo de plataforma hasta que el wizard se completa (`isProfileComplete`).
- Avatar guardado como `data:image/...;base64` directamente en DB (compatible con el filesystem efímero de Render).

8.5. “Mis actividades”

- Smart sort: pendientes primero (por fecha asc), luego sin fecha, luego completadas.
- Badge **HOY** y resaltado rojo si está vencida.
- Botón **Marcar completa** con CTA gradient.
- Link directo a la sala / meeting URL.
- Barra de progreso personal.

9. Despliegue

9.1. Render (`render.yaml`)

```
services:
  - type: web
    name: inspiratoria-backend
    env: python
    buildCommand: pip install -r backend/requirements.txt && \
                  python backend/manage.py collectstatic --noinput && \
                  python backend/manage.py migrate --noinput
    startCommand: cd backend && uvicorn mentorloop_clone.asgi:application \
                  --host 0.0.0.0 --port $PORT

  - type: web
    name: inspiratoria-frontend
    env: node
    buildCommand: cd frontend && npm install && npm run build
    startCommand: cd frontend && npm start

databases:
  - name: inspiratoria-db
    plan: free
    databaseName: inspiratoria
```

- Auto-deploy desde rama `main` (push → build → deploy).
- Postgres free 90 días → ~USD 7/mes.
- Dominio productivo: `inspiratoria.aplifully.com` (CNAME al servicio Render).

9.2. Local

```
./start_all.sh          # backend 8001 + frontend 3000 + tail logs
./start_backend.sh      # solo backend
```

`start_all.sh` libera puertos, lanza ambos servicios con `nohup`, espera `healthcheck` y muestra `tail -f` de ambos logs (`/tmp/inspiratoria_*.log`).

10. Variables de entorno

Backend

Variable	Descripción
DJANGO_SECRET_KEY	Auto-generada en Render
DEBUG	False en prod
ALLOWED_HOSTS	inspiratoria-backend.onrender.com, inspiratoria.aplify.com
CORS_ALLOWED_ORIGINS	Dominios frontend separados por coma
DATABASE_URL	Postgres (auto en Render)
EMAIL_HOST_USER / EMAIL_HOST_PASSWORD	SMTP Gmail
GEMINI_API_KEY	Gemini 2.5 Flash
FRONTEND_URL	Para construir links absolutos en emails
LINKEDIN_CLIENT_ID / LINKEDIN_CLIENT_SECRET	OAuth LinkedIn

Frontend

Variable	Descripción
NEXT_PUBLIC_API_BASE_URL	URL absoluta del backend
NEXT_PUBLIC_LINKEDIN_REDIRECT_URI	Callback OAuth

11. Comandos y scripts

Comando	Para qué
./start_all.sh	Backend + frontend en local
./start_backend.sh	Solo backend (uvicorn --reload)
python backend/manage.py migrate	Aplica migraciones
python backend/manage.py createsuperuser	Crea super usuario Django
python backend/seed_intelligent_match_demo.py	Pobla mentores/mentees demo
python backend/seed_all_roles.py	Pobla todos los roles
python backend/clear_all_keep_admin.py	Limpia DB conservando admin
cd frontend && npm run dev	Dev server Next.js
cd frontend && npm run build	Build de producción
cd frontend && npx next build	Verificación de tipos + bundle

12. Histórico de commits relevantes

```
80545ed feat(portal/modulos): sync activity dates from cronograma + show orphan
activities
819a01a feat(portal): skeleton loaders + lock invitation fields in profile wizard
fae3f33 feat: PM Console v2 + participant management + portal sync
81e2540 feat: mentee onboarding wizard – 4-step profile, block access until complete
301809c feat: mentee dashboard, role-based nav, my-mentor view, mentee sessions
a7aedef4 feat: session audit log, email notifications mentor/mentee, SVG icons
f725a79 fix: move useState hooks before early returns (React #310)
e1219fd fix: session creation – load mentees from programs, add form validation feedback
833e416 feat: mentoring sessions, mentee profiles, activity completion, network, AI
suggestions
06dccc8 feat: multi-step mentor profile wizard (4 pasos, DB-backed)
f12e385 feat: lock platform navigation until profile is complete
1b71c46 fix: wrap useSearchParams in Suspense for core & studio pages
f6237d6 fix: resolve all TypeScript build errors for Render deploy
5f85f1a feat: chat profile detail modal – click avatar to view participant profile
19balc0 feat: chat system with dark theme, badges, ecosystem, profile & full portal
88c3fc3 feat: avatar guardado como base64 en DB – persistente en Render
17df3c0 fix: agregar roles admin_root, coordinator, project_manager, billing
d8881ea fix: add FRONTEND_URL setting for activation emails + CORS for aplify domain
```

13. Glosario

- **PM Console / Studio** — Workspace `/studio/{slug}/program/{id}` desde el que el Project Manager diseña, agenda y ejecuta el programa.
 - **Portal** — Vista de cada participante en `/p/{portal_code}/...`, 100 % sincronizada con la consola PM.
 - **Cronograma** — Conjunto de `Activity` con `start_date/end_date` agendadas, fuente única de verdad de fechas en el portal.
 - **OTP** — Código de un solo uso (4 dígitos, 15 min) enviado por email para login.
 - **TOTP** — Segundo factor opcional con Google/Microsoft Authenticator.
 - **Portal code** — Identificador corto único por usuario para deep-links de participante.
 - **Smart match** — Endpoint que ejecuta el algoritmo 6D + Gemini opcional.
 - **Self-enroll** — Auto-inscripción pública vía `/enroll/{programId}` sin auth.
-

*Mantén este archivo como **single source of truth** del proyecto. Cuando cambie arquitectura, modelos o flujos críticos, actualiza la sección correspondiente y commitea junto con el cambio.*